DESIGN AND ANALYSIS OF ALGORITHMS

Dijkstra's Algorithm

Presented by

Riddhi Thorat - 25070521246

Symbiosis Institute of Technology Nagpur

CORE IDEA

Shortest paths

One greedy choice at a time.

ORIGIN AND LEGACY

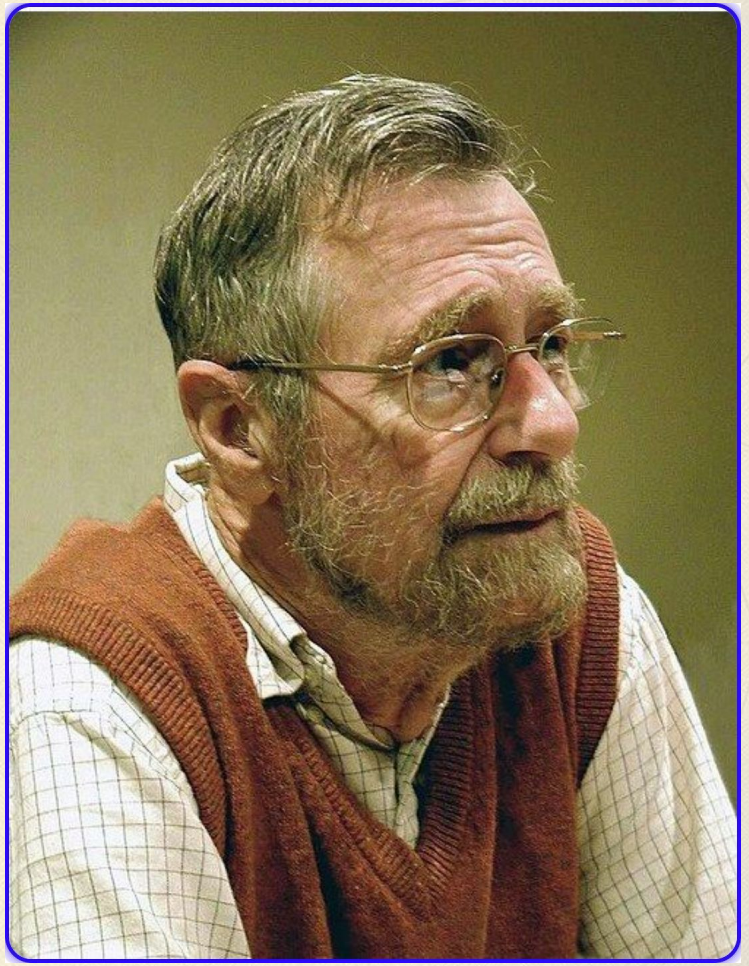
From a 1956 idea to a classic algorithm

Designed in 1956 and published in 1959 by Dutch computer scientist Edsger W. Dijkstra.

He won the Turing Award in 1972.

The Edsger W. Dijkstra Paper Prize in Distributed Computing carries his name.

> “Computer Science is no more about computers than astronomy is about telescopes.”



Made with GAMMA

PROBLEM AND MODEL

Shortest paths, one greedy choice at a time

Dijkstra solves a single-source shortest path problem.



Input

Weighted graph G = {V, E}, non-negative edge weights, and a source vertex.



Output

Shortest distance from the source to every other vertex.



Example

Cities are vertices; driving distances are edge weights.

Unlike an MST: Dijkstra minimizes distance from one source; an MST minimizes total tree weight.

Made with GAMMA

The workflow: initialize, choose, relax, repeat

1. Set source distance to 0; all others to ∞.
2. Keep a visited set.
3. Pick unvisited node with smallest distance.
4. Mark it visited.
5. Relax every neighbor using d[u] + weight.
6. Update when the new distance is smaller.
7. Repeat until all nodes are visited.

SHORT PSEUDOCODE

```
dist[s] = 0
while unvisited:
  u = minDist()
  visit(u)
  for each v:
    relax(u, v)
```

Made with GAMMA

PERFORMANCE

Complexity depends on the data structure

Implementation	Time	Space
Adjacency matrix	$O(V^2)$	O(V)
Priority queue / min-heap	O((V + E) log V)	O(V)
Sparse-graph shorthand	O(E log V)	O(V)

Use the matrix for dense graphs; use a min-heap to prioritize the next closest vertex.

Dijkstra's Algorithm Complexity

- **Time Complexity:** $O(V^2)$

With min priority queue time complexity is O(V + E Log V)

E = Number of edges and
V = Number of vertices

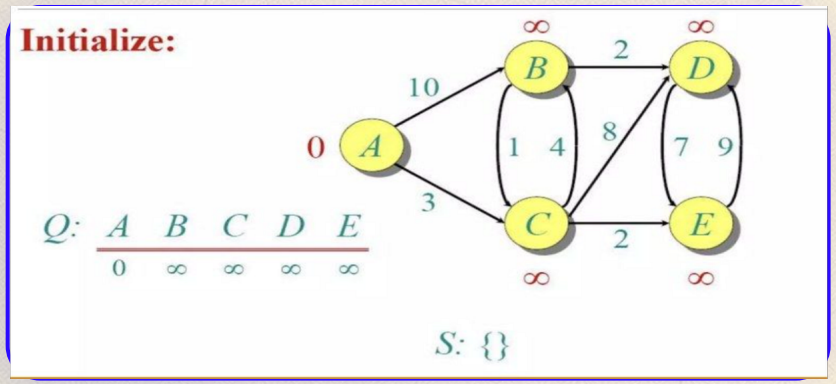
- **Space Complexity:** $O(V)$

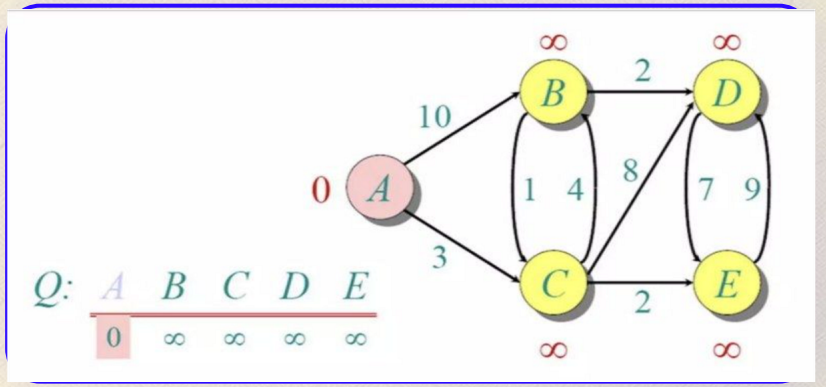
Made with GAMMA

Worked example: the first labels emerge

Start at A with d(A)=0 and all other distances at ∞.

- Visit A: relax A→B = 10 and A→C = 3.
- Visit C next: improve B to 7; set D to 11 and E to 5.
- Keep the smallest tentative label as the next greedy choice.





Made with GAMMA

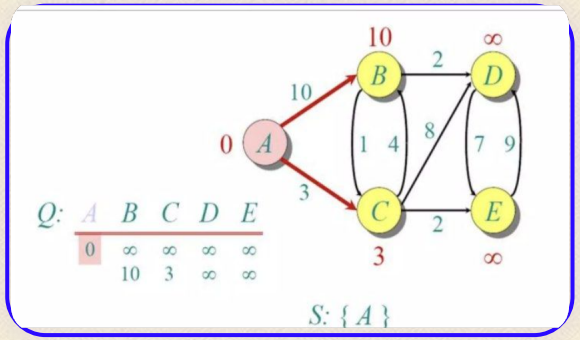
Worked example: finalize distances and paths

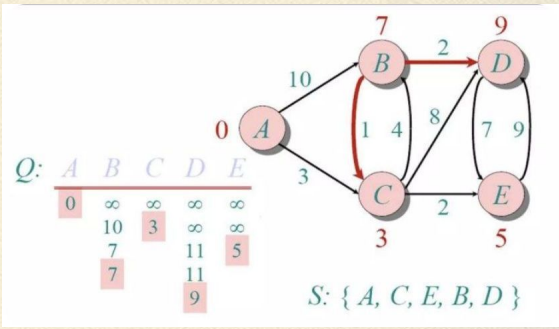
After C, E, B, and D are selected, every shortest label is final.

Final distances from A: A=0, C=3, E=5, B=7, D=9.

Shortest-path tree: A→C, C→E, C→B, B→D.

The predecessor links explain not only how far, but how to reach each destination.





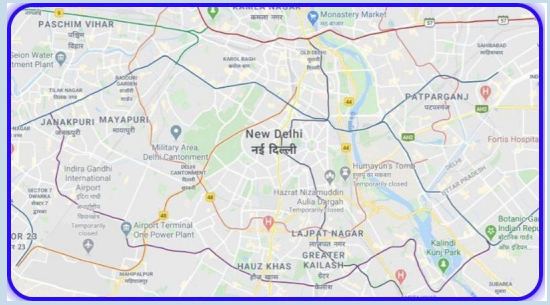
Made with GAMMA

APPLICATIONS

Applications: maps and routing

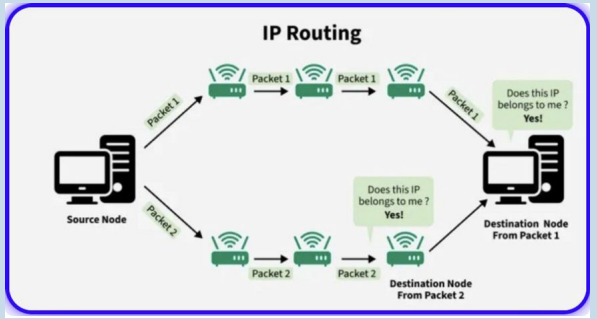
Digital mapping

Junctions = vertices; **roads** = edges; time or distance = weights. Live traffic triggers recalculation.



IP routing

Routers = vertices; network links = edges. OSPF updates routing tables using shortest paths.



Made with GAMMA

Applications: people and telephone networks

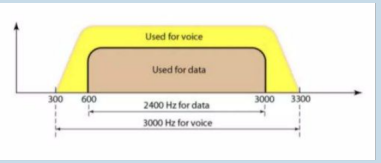
Social networking

People are vertices; relationships are edges; degrees of connection support “People You May Know.”



Telephone networks

Towers or exchanges are vertices; cables are edges; signal delay is the weight; shortest paths support bandwidth optimization.



Made with GAMMA

WHEN TO USE IT

Use it when weights are non-negative

Advantages

Guaranteed shortest path; works on directed and undirected graphs; finds all destinations in one run; fast with a priority queue.

Disadvantages

Fails with negative edges; blind search can explore many nodes; needs O(V) memory; can be slow on dense graphs; must be rerun when the graph changes.

TAKEAWAY

Dijkstra turns local greedy choices into globally shortest paths.

Made with GAMMA

Thank You

Made with GAMMA

Made with GAMMA